

```
from mcmc import *
from dram import *
def adaptive_metropolis_mcmc(q0, J_func, r_calc, M, rng_seed=None):
    """
    Metropolis-Hastings MCMC with Adaptive Metropolis (AM)

    Parameters:
    q0: initial sample
    J_func: initial proposal distribution function (for first 100 iterations)
    r_calc: function to compute acceptance ratio r(q*, q_{k-1})
    M: number of samples to generate
    rng_seed: random seed for reproducibility

    Returns:
    samples: array of MCMC samples
    acceptance_rate: acceptance rate of the chain
    Vk_at_101: the covariance matrix computed at k=101 (for problem 2.1)
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    p = len(q0) # dimension of parameter space (should be 2)
    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, p))
    samples[0] = q_current

    # AM parameters
    sp = 2.38**2 / p # scaling factor (2.38^2/p)
    k0 = 100 # start adapting after 100 samples

    accepted = 0
    Vk_at_101 = None # to store the covariance at k=101 for problem 2.1

    print("Running Adaptive Metropolis MCMC...")
    for i in range(1, M):
        # Step 2.1: Generate proposal
        if i < k0:
            # First k0 iterations: use fixed proposal J_func
            q_proposed = J_func(q_current)
        else:
            # Adaptive Metropolis proposal
            if i == k0:
                # First time using AM (k=101)
                # Compute covariance from samples[0:k0] (first 100 samples)
                samples_so_far = samples[:k0]
                Vk = np.cov(samples_so_far, rowvar=False)
                Vk_at_101 = Vk.copy() # Save for problem 2.1
                print(f"\nAt k=101, computed Vk from first {k0} samples:")
                print(f"Vk = {Vk}")

                # Use this Vk for the proposal at k=101
                proposal_cov = sp * Vk
            elif np.mod(i, k0)==1:
                # For subsequent iterations, compute covariance from all samples up to i
                samples_so_far = samples[:i]
                Vk = np.cov(samples_so_far, rowvar=False)
                proposal_cov = sp * Vk

            # Generate proposal from N(q_current, sp * Vk)
            q_proposed = np.random.multivariate_normal(q_current, proposal_cov)

        # Step 2.2: Compute the ratio r(q*, q_{k-1})
        r = r_calc(q_proposed, q_current)

        # Step 2.3: Accept with probability min(1, r)
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        samples[i] = q_current

        if (i + 1) % 100 == 0:
            print(f" Completed {i + 1}/{M} iterations, acceptance rate: {accepted/(i):.3f}")

    acceptance_rate = accepted / (M - 1)

    return samples, acceptance_rate, Vk_at_101, Vk

def run_adaptive_metropolis_for_heat_equation(M=None, func = None, k_for_am = None, burn = None, title = 'AM', data_filename = 'HW06_Problem1.mat'):
    """
    Run Adaptive Metropolis algorithm for the heat equation problem (Problem 1.1)
    """
    # Setup problem
    rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_filename)

    # Initial proposal covariance (same as in Problem 1.1)
    D = np.array([[1e-2, 0],
                  [0, 2e-10]])

    # Create initial proposal function J(q* | q_{k-1}) = N(q_{k-1}, D)
    # This will be used for the first 100 iterations
    def initial_proposal_func(q_current, returnd=False):
        return (
            np.random.multivariate_normal(q_current, D),
            D
        ) if returnd else np.random.multivariate_normal(q_current, D)
    # Create function to compute r(q*, q_{k-1})
    posterior_ratio_func = create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0)

    if M is None:
        M = 1000

    print(f"\nRunning Adaptive Metropolis algorithm for {M} iterations...")
    print(f"Initial proposal distribution (first {100} iterations): N(q_prev, D) with D = diag([{D[0,0]}, {D[1,1]}])")
    print(f"AM parameters: sp = 2.38^2/{len(q0)} = {2.38**2/len(q0):.4f}, k0 = 100")
    if func == 'dram':
        if data_filename is None:
            data_filename = 'HW06_Problem3.mat'

        rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_filename)
        ns=0.01; sigma2_s=4.0

        samples, acceptance_rate, Vk_at_101, Vk_at_end = dram(
            q0 = q0, s2_0 = sigma2_0,
            J_func = initial_proposal_func,
            r_calc = posterior_ratio_func,
            M= M, ns = ns, sigma2_s = sigma2_s, n_obs = len(T_obs), rod = rod, x_obs =x_obs, T_obs = T_obs,
            rng_seed=42, k = k_for_am
        )
        print("am,san", samples.shape)

    elif func == 'drm':
```

```

    samples, acceptance_rate, Vk_at_101, Vk_at_end = drm(
        q0,
        initial_proposal_func,
        posterior_ratio_func,
        M,
        rng_seed=42
    )
else:
    samples, acceptance_rate, Vk_at_101, Vk_at_end = adaptive_metroplis_mcmc(
        q0,
        initial_proposal_func,
        posterior_ratio_func,
        M,
        rng_seed=42
    )

print(f"\nFinal acceptance rate: {acceptance_rate:.3f}")

# Problem 2.1: Report Vk at k=101
print(f"\n{' '*60}")
print("PROBLEM 2.1 RESULTS")
print(f"\n{' '*60}")
print(f"\nAt k=101, the new Vk generated from the first 100 samples is:")
print(f"Vk = {Vk_at_101}")

# Problem 2.2: Report Vk at final k
print(f"\n{' '*60}")
print("PROBLEM 2.2 RESULTS")
print(f"\n{' '*60}")
print(f"\nAt k=M, the final Vk generated is:")
print(f"Vk = {Vk_at_end}")

print("\nComputing posterior values for MAP estimate...")

# Prior parameters
prior_Phi_mean = -15
prior_Phi_std = 10
prior_h_mean = 0.001
prior_h_std = 0.005

# Compute unnormalized posterior for each sample
print("\nComputing posterior values for MAP estimate...")
posterior_values = np.zeros(M)
if func == "dram": # unknwon sigma**2
    # Find MAP estimate (maximum unnormalized posterior with sigma_squared)

    print("SHAPE", samples.shape)
    q_samples, s2_samples = samples[:, :-1], samples[:, -1]
    q_samples_post = q_samples[burn:]
    s2_samples_post = s2_samples[burn:]
    posterior_values = np.zeros(len(q_samples_post))
    for i, (q, s2) in enumerate(zip(q_samples_post, s2_samples_post)):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood with current sigma_squared
        n = len(x_obs)
        likelihood = (1/np.sqrt(2*np.pi*s2))**n * np.exp(-0.5 * np.sum((T_obs - T_model)**2 / s2))

        # Prior for q
        prior_q = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                    norm.pdf(h, prior_h_mean, prior_h_std))

        # Prior for sigma_squared (Inverse Gamma)
        # p(sigma_squared) \propto (sigma_squared)^(-(ns/2+1)) * exp(-(ns*sigma_squared_s)/(2sigma_squared))
        log_prior_sigma = -(ns/2 + 1) * np.log(s2) - (ns * sigma2_s) / (2 * s2)
        prior_sigma = np.exp(log_prior_sigma)

        # Unnormalized posterior
        posterior_values[i] = likelihood * prior_q * prior_sigma

    map_idx = np.argmax(posterior_values)
    q_map = q_samples_post[map_idx]
    s2_map = s2_samples_post[map_idx]
    post_map = posterior_values[map_idx]

else:
    for i, q in enumerate(samples):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood
        n = len(x_obs)
        likelihood = np.exp(-0.5 * np.sum((T_obs - T_model)**2 / sigma2_0))

        # Prior
        prior = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                  norm.pdf(h, prior_h_mean, prior_h_std))

        # Unnormalized posterior (pi(data|q)pi0(q))
        posterior_values[i] = likelihood * prior

    # Find MAP estimate (maximum unnormalized posterior)
    map_idx = np.argmax(posterior_values)
    q_map = samples[map_idx]
    post_map = posterior_values[map_idx]

print(f"\n{' '*60}")
print("PROBLEM 1.1 RESULTS")
print(f"\n{' '*60}")
print(f"\nMAP Estimate (q_MAP):")
print(f"  O = {q_map[0]:.6f} W/cmBI")
print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
print(f"  P(P...|q_MAP)P0(q_MAP) = {post_map:.2e}")

print(f"\nPosterior Distribution P(q|P...) Summary:")
print(f"\n{' '*60}")
print(f"O:")
print(f"  Mean = {np.mean(samples[:, 0]):.6f} W/cmBI")
print(f"  Std = {np.std(samples[:, 0]):.6f} W/cmBI")
print(f"  95% CI = [{np.percentile(samples[:, 0], 2.5):.6f}, {np.percentile(samples[:, 0], 97.5):.6f}])")

print(f"\nh:")
print(f"  Mean = {np.mean(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  Std = {np.std(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  95% CI = [{np.percentile(samples[:, 1], 2.5):.6e}, {np.percentile(samples[:, 1], 97.5):.6e}])")

```

```

# Create diagnostic plots - 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(14, 12))

# Plot 1: Phi histogram (top left)
axes[0, 0].hist(samples[:, 0], bins=30, density=True, alpha=0.7, color='skyblue', edgecolor='black')
axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[0]:.4f}')
axes[0, 0].set_xlabel('O (W/cmBI)')
axes[0, 0].set_ylabel('Density')
axes[0, 0].set_title('Marginal Posterior  $\Pi_{\Phi}(O|\Pi_{\dots})$ ')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Plot 2: h histogram (top right)
axes[0, 1].hist(samples[:, 1], bins=30, density=True, alpha=0.7, color='lightcoral', edgecolor='black')
axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[1]:.6f}')
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior  $\Pi_{\Phi}(h|\Pi_{\dots})$ ')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Joint posterior scatter (bottom left)
axes[1, 0].scatter(samples[:, 0], samples[:, 1], alpha=0.5, c='purple', s=10)
axes[1, 0].plot(samples[:, 0], samples[:, 1], alpha=0.5, c='purple')
axes[1, 0].scatter(q_map[0], q_map[1], color='Magenta', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('O (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples  $\Pi_{\Phi}(O, h|\Pi_{\dots})$ ')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Model fit with MAP parameters (bottom right)
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
p = f'{title}.png'
os.path.exists(p)
plt.savefig(p, dpi=150)
plt.show()

return samples, posterior_values, q_map, Vk_at_101, acceptance_rate

```